Ingegneria del Software — Soluzione del Tema 11/06/2024

Esercizio 1

Si consideri la seguente classe `JobManager` per gestire componenti software (`Worker`) che eseguono computazioni (`Job`). Ogni `Worker` ha una coda di `Job` in attesa di essere eseguiti, ottenibile mediante il metodo `pendingJobs`.

Gli utilizzatori di `JobManager` possono inviare un nuovo `Job` mediante il metodo `submit`. Se esiste almeno un `Worker` con meno di 10 `Job` in coda, allora il `Job` sottomesso viene assegnato al primo worker nella lista dei worker avente meno di 10 `Job` in coda, altrimenti viene sollevata l'eccezione `TooManyJobsException`. Il metodo `submit` richiede che il nuovo `Job` non sia tra quelli "pending" nella coda di un qualche `Worker`.

Il metodo `balance` viene invocato per bilanciare le code dei `Worker` in maniera tale che ognuno abbia lo stesso numero di `Job` in coda (con una differenza di al più un `Job`).

```
public class JobManager {
  // Ritorna la lista di worker attivi.
  public /*@ pure @*/ List<Worker> workers() { .. }

  // Ritorna la coda di job associati al worker passato come parametro.
  // Ritorna la lista vuota se il worker non ha job in coda.
  public /*@ pure @*/ List<Job> pendingJobs(Worker worker) { .. }

  // Invia il job passato per parametro per essere eseguito da un worker.
  // Se esiste almeno un worker con meno di 10 job in coda, allora il job sottomesso
  // viene assegnato al primo worker nella lista dei worker avente meno di 10 job in coda,
  // altrimenti viene sollevata l'eccezione TooManyJobsException.
  // Il metodo richiede che il nuovo job non sia tra quelli in coda a qualche worker.
  public void submit(Job job) throws TooManyJobsException;

  // Bilancia le code dei worker in maniera tale che ognuno abbia lo stesso numero di
  // job in coda (con una differenza al piu' di un job per tenere conto del
  // caso in cui il numero totale di job non sia divisibile per il numero totale di worker).
  public void balance();
}
```

Domanda a)

Si specifichi in JML il metodo `submit`.

Soluzione

Definiamo i seguenti predicati, che utilizzeremo nelle specifiche dei metodi.

```
//@ unchangedWorkers() <==> workers().size() == \old(workers().size()) &&
//@   (\forall int i; i>=0 && i<workers().size(); workers().get(i)==\old(workers().get(i)))
//@
//@ unchangedWorker(w) <==> pendingJobs(w).size() == \old(pendingJobs(w).size()) &&
//@   (\forall int i; i>=0 && i<pendingJobs(w).size();
//@     pendingJobs(w).get(i)==\old(pendingJobs(w).get(i)))
```

Definiamo quindi `submit` come segue.

```
//@ requires job != null &&
//@   (\forall Worker w; workers().contains(w); !pendingJobs(w).contains(job))
//@
//@ ensures unchangedWorkers() &&
//@   (\exist int i; i>=0 && i<workers().size(); \old(pendingJobs(workers().get(i)).size()<10) &&
//@     (\forall int j; j>=0 && j<i; \old(pendingJobs(workers().get(j)).size()==10) &&
//@       unchangedWorker(workers().get(j))) &&
//@     (\forall int j; j>i && j<workers().size(); unchangedWorker(workers().get(j))) &&
//@     pendingJobs(workers().get(i)).size() == \old(pendingJobs(workers().get(i)).size())+1 &&
//@     pendingJobs(workers().get(i)).get(pendingJobs(workers().get(i)).size()-1).equals(job) &&
//@     (\forall int k; k>=0 && k<\old(pendingJobs(workers().get(i)).size());
//@       pendingJobs(workers().get(i)).get(k).equals(\old(pendingJobs(workers().get(i)).get(k))))
//@
//@ signals (TooManyJobsException e)
//@   (\forall int i; i>=0 && i<workers().size(); \old(pendingJobs(workers().get(i)).size()==10) &&
//@   unchangedWorkers() &&
//@   (\forall int i; i>=0 && i<workers().size(); unchangedWorker(workers().get(i)))
public void submit(Job job) throws TooManyJobsException;
```

Domanda b)

Si specifichi in JML il metodo `balance`.

Soluzione

```
//@ ensures unchangedWorkers() &&
//@   (\forall int i; i>=0 && i<workers().size();
//@     (\forall int j; j>=0 && j<workers().size();
//@       Math.abs(pendingJobs(workers().get(i)).size()-pendingJobs(workers().get(j)).size())<=1)) &&
//@   (\forall Job j; (\exist Worker w; workers().contains(w) && \old(pendingJobs(w).contains(j)));
//@     (\exist Worker w1; workers().contains(w1) && pendingJobs(w1).contains(j))) &&
//@   (\forall Job j; (\exist Worker w; workers().contains(w) && pendingJobs(w).contains(j));
//@     (\exist Worker w1; workers().contains(w1) && \old(pendingJobs(w1).contains(j)))) &&
public void balance()
```

Domanda c)

Si consideri la seguente implementazione, che fa uso di una lista di `Worker` `workers` e di una mappa `jobsByWorker` per mantenere le informazioni relative ai `Job` in attesa.

```
public class JobManager {
  private List<Worker> workers;
  private Map<Worker, List<Job>> jobsByWorker;
... }
```

Per tale implementazione si fornisca l'invariante di rappresentazione.

Soluzione

L'invariante di rappresentazione specifica che la lista di worker non è nulla, non contiene worker nulli e coincide con l'insieme di chiavi della mappa (non contiene quindi duplicati). Inoltre la mappa non è nulla, non contiene valori (liste) nulle e le liste non devono contenere elementi nulli e devono avere dimensione minore di 10. Infine non devono esserci job duplicati.

```
//@ private invariant
//@ workers != null && !workers.contains(null) &&
//@ workers.containsAll(jobsByWorker.keySet()) &&
//@ jobsByWorker.keySet().containsAll(workers) &&
//@ jobsByWorker != null &&
//@ (\forall Worker w; jobsByWorker.containsKey(w);
//@    jobsByWorker.get(w) != null && !jobsByWorker.get(w).contains(null) &&
//@    jobsByWorker.get(w).size() <= 10 &&
//@    (\forall int i; i>=0 && i<jobsByWorker.get(w).size();
//@      (\forall int j; j>i && j<jobsByWorker.get(w).size();
//@        !jobsByWorker.get(w).get(i).equals(jobsByWorker.get(w).get(j)) ) ) ) &&
//@ (\forall Worker w; jobsByWorker.containsKey(w);
//@   (\forall Worker w1; jobsByWorker.containsKey(w1) && !w1.equals(w);
//@     !(\exists Job j; ; jobsByWorker.get(w).contains(j) && jobsByWorker.get(w1).contains(j) ) ) )
```

Domanda d)

Si consideri una classe `JobManager1` che aggiunge ai metodi di `JobManager` il metodo `pack` che assegna incondizionatamente al primo `Worker` della lista tutti i `Job` in coda.

Può `JobManager1` essere definito come estensione di `JobManager` in accordo con il principio di sostituzione? Si motivi la propria risposta.

Soluzione

`JobManager1` *non* può essere definito come estensione di `JobManager` secondo il principio di sostituzione, in quanto il nuovo metodo andrebbe a invalidare l'invariante pubblico secondo cui non possono esistere `Worker` con più di 10 `Job` in coda.

Esercizio 2

Si consideri la seguente implementazione (parziale) dei tipi `JobManager`, `Worker` e `Job` dell'esercizio precedente.

```
public class JobManager {
  private List<Worker> workers;
  private Map<Worker, List<Job>> jobsByWorker;

  public JobManager(int numWorkers) {
    workers = new ArrayList<Worker>();
    jobsByWorker = new HashMap<Worker, List<Job>>();
    // riempie la lista Workers con numWorkers elementi e la mappa jobsByWorker
    // associando ad ogni worker una lista vuota di Job. Fa quindi partire i thread associati
    // ad ogni worker.
  }

  public List<Worker> workers() { .. }

  public List<Job> pendingJobs(Worker worker) { .. }

  public void submit(Job job) throws TooManyJobsException {
    boolean added = false;
    for(int i=0; i<workers.size() && ! added; i++) {
      List<Job> l = jobsByWorker.get(workers.get(i));
      synchronized(l) {
        if(l.size()<10) {
          l.add(job);
          l.notify();
          added = true;
        }
      }
    }
    if(!added) throw new TooManyJobsException();
  }
}

class Worker implements Runnable {
  private List<Job> jobs;
  public Worker(List<Job> jobs) {
    this.jobs = jobs;
  }
  @Override
  public void run() {
    while(true) {
      Job j = null;
      synchronized(jobs) {
        while(jobs.isEmpty()) {
          try {jobs.wait();}
          catch(Exception ex) {ex.printStackTrace();}
        }
        j = jobs.remove(0);
      }
      j.doJob();
    }
  }
}

interface Job {
  public void doJob();
}
```

Domanda a)

L'implementazione fornita, con particolare riguardo al metodo `submit` di `JobManager` e alla classe `Worker`, sono correttamente sincronizzate? Si motivi la propria risposta e in caso negativo si spieghi quale parte di codice andrebbe modificata e come per risolvere gli eventuali problemi identificati.

Soluzione

L'implementazione fornita è corretta. Si osservi in particolare che il metodo `submit` accede alla mappa `jobsByWorker` senza sincronizzarsi ma questo non costituisce un problema in quanto la mappa risulta di fatto immutabile. Sono invece mutabili le liste di `Job` associate ad ogni worker e per questo vi si accede sincronizzandosi opportunamente, tanto nel metodo `submit`, quanto nel metodo `run` di `Worker`.

Domanda b)

Si completi l'implementazione del costruttore della classe `JobManager`, sostituendo al commento il codice corrispondente.

Soluzione

```
public JobManager(int numWorkers) {
  workers = new ArrayList<Worker>();
  jobsByWorker = new HashMap<Worker, List<Job>>();
  // ...
  for(int i=0; i<numWorkers; i++) {
    List<Job> jobs = new ArrayList<Job>();
    Worker w = new Worker(jobs);
    workers.add(w);
    jobsByWorker.put(w, jobs);
    new Thread(w).start();
  }
}
```

Esercizio 3

Domanda a) Si consideri la dichiarazione del seguente metodo Java:

```
public static List<Integer> zip(List<Integer> first, List<Integer> second)
```

Il metodo restituisce una lista in cui alterna gli elementi della prima e della seconda lista, interrompendosi quando una delle due liste è esaurita. Per esempio, se first è la lista [1, 3, 5] e second è la lista [2, 4], il risultato è la lista [1, 2, 3, 4].

Scrivere il corpo del metodo utilizzando esclusivamente i costrutti della programmazione funzionale di Java.

Suggerimento: Si può utilizzare il metodo `IntStream.range()`, che restituisce uno stream di interi del range passato come argomento. Ad esempio, `IntStream.range(0,100)` restituisce lo stream $[0, 1, 2, ..., 99]$.

Soluzione

```
public static List<Integer> zip(List<Integer> first, List<Integer> second) {
  return IntStream.range(0, Math.min(first.size(), second.size()) * 2)
    .mapToObj(i -> i % 2 == 0 ? first.get(i / 2) : second.get(i / 2))
    .collect(Collectors.toList());
}
```

Domanda b)

Si considerino le seguenti classi Java, in cui sono riportate solo le parti rilevanti:

```
public class Studente {
  ...
  public Optional<Tesi> getTesi() {...}
}

public class Tesi {
  ...
  public Optional<Controrelatore> getControrelatore() {...}
}

public class Controrelatore {
  ...
  public Optional<String> getEnte() {...}
}
```

Le classi descrivono la situazione di uno studente laureando, che potrebbe avere consegnato la tesi di laurea (oppure no); la tesi potrebbe avere un controrelatore; nel caso in cui questo sia presente ma sia un esterno, il controrelatore ha un'affiliazione, ossia un ente (p. es. un'altra unversità).

Si consideri il seguente metodo, che restituisce l'ente di affiliazione del controrelatore della tesi di uno studente; qualora l'ente o il controrelatore o la tesi non siano presenti, restituisce "nessun ente".

Scrivere il corpo del metodo utilizzando esclusivamente i costrutti della programmazione funzionale di Java.

```
public static String getEnteControrelatore(Studente studente){
  return ...
```

Soluzione

```
    public static String getEnteControrelatore(Studente studente) {
        return Studente::getTesi  //returns Optional<Tesi>
        .flatMap(Tesi::getControrelatore)  //returns Optional<Controrelatore>
        .flatmap(Controrelatore::getEnte)  //returns Optional<String>
        .orElse("nessun ente"); //returns the String in the Optional if any, otherwise "nessun ente"
    }
```

Esercizio 4

Si consideri il seguente metodo:

```
1  public static int prod(int n, int[] x) {
2    if (x == null || x.length < n || n <= 0) {
3      return 0;
4    }
5    int p = 1;
6    int i = 0;
7    while (i < n) {
8      if (x[i] > 1 || x[i] < -1) {
9         p = p * x[i];
10     }
11     i++;
12   }
13   return p;
14 }
```

Domanda a)

Si determini un insieme *minimo* di test per la coppia (n, x) per garantire la copertura delle istruzioni (statement coverage).

Soluzione

Servono due casi: $(-, null)$ e $(1, \{2\})$. L'insieme è minimo in quanto ci sono due return. Il "$-$" sta per un valore qualunque.

Domanda b)

Si determini un insieme *minimo* di test per la coppia (n, x) per garantire la copertura delle decisioni (edge coverage).

Soluzione

I due casi precedenti non coprono la decisione negativa del secondo if. Basta prendere per es. $(-, null)$ e $(2, \{2, 1\})$. Ancora due casi bastano, sfruttando le iterazioni del loop.

Domanda c)

Si determini un insieme *minimo* di test per la coppia (n, x) per garantire la copertura delle decisioni e delle condizioni (edge and condition coverage).

Soluzione

Per coprire le condizioni del primo if occorre prendere 3 casi: $(-, null)$, $(1, \{\})$, $(0, \{1\})$.

Per coprire le condizioni del secondo if occorre prendere anche: $(3, \{2, 1, -2\})$.

Anche tutte le decisioni sono coperte. Quindi servono 4 casi.